# Loan Default Prediction

## Comparing Classic ML and Neural Networks under Class Imbalance

Minwoo Yoo  Nathaniel Badalov

Final Project Group 15 · Home Credit Default Risk (Kaggle)

# Business Problem and Motivation

## Goal

Predict the probability that a loan applicant will default before making a lending decision.

## Why it matters

- False negatives: approve borrowers who default
- False positives: reject creditworthy borrowers
- Models help balance risk vs. profitability

**Key challenge**
Default events are rare (~8%) $\Rightarrow$ strong class imbalance

# From Imbalance to Research Question

**Why imbalance matters**

- Models can achieve high ROC AUC but still predict no defaults
- Default threshold (0.5) often fails in imbalanced settings

**Common solutions**

- Reweighting the loss (class weights)
- Resampling the data (e.g., SMOTE)
- Adjusting the decision threshold

## Research Question

- Do neural networks respond to class imbalance handling techniques the same way as classic shallow learners?

**Models compared**

- Logistic Regression, Random Forest, HistGradientBoosting
- Shallow and Deep MLP

# Dataset Overview

## What the dataset represents

- Loan applications from Home Credit
- Focus on borrowers with limited credit history
- Tabular structure (Each row = one applicant)
- Goal: predict default risk

## Main feature groups

- Demographics (age, family status)
- Employment and income
- Loan details (amount, annuity)
- External credit scores: `EXT_SOURCE_*`

# Data Structure and Class Imbalance

## Dataset characteristics

- Train data: **307,511**
- Test data: **48,744**
- Features: **122**

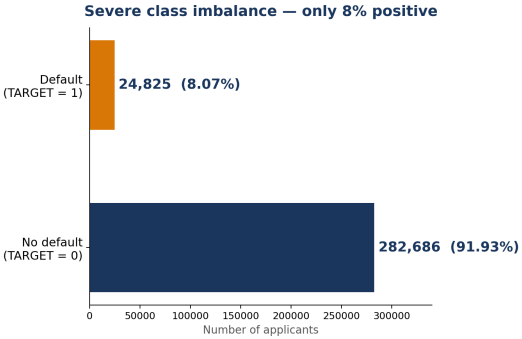
## Target variable

- 1: default
- 0: repaid

## Class imbalance

Defaults: only **8.07%** default

$\Rightarrow$ severe class imbalance



# Classic ML Strategy

**Why start with classic ML?**

- The data is tabular: rows are applicants, columns are features
- Classical models are strong baselines for tabular prediction
- They provide a benchmark for the neural networks

**Models tested**

- **Logistic Regression**: strong baseline for probability prediction
- **Random Forest**: captures nonlinear patterns and reduces overfitting
- **Histogram Gradient Boosting**: captures complex interactions efficiently

## Handling class imbalance

- Logistic Regression and Random Forest give more weight to default cases (`class_weight='balanced'`).
- Histogram Gradient Boosting used without class weights (boosting still captures rare cases well).

# Classic ML Results

| Model | Best hyperparameters | Val ROC AUC |
|---|---|---|
| HistGradientBoosting | `learning_rate`=0.05; `max_iter`=200; `leaf`=100 | **0.7595** |
| Logistic Regression | `C`=0.1; `tol`=1e-5; `class_weight=balanced` | 0.7487 |
| Random Forest | `n_estimators`=200; `min_leaf`=1; `balanced` | 0.7470 |

**Key takeaways**

- Histogram Gradient Boosting performs best (0.7595), suggesting boosted trees capture nonlinear interactions in tabular data effectively.
- Logistic Regression and Random Forest perform similarly, indicating strong linear signal alongside moderate nonlinearities.
- Random Forest shows signs of overfitting (train 0.96 vs val 0.747), while boosting remains more stable.

# Neural Network Models

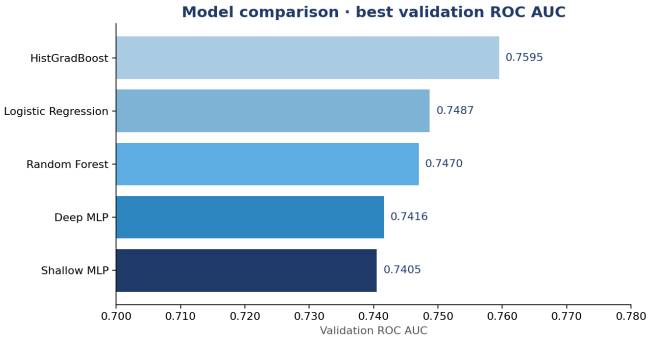
| Architecture | Hidden | Parameters | Val AUC |
|---|---|---|---|
| Shallow MLP | $(64, )$ | $\sim$15.7K | 0.7405 |
| Deep MLP | $(128, 64, 32)$ | $\sim$42K | **0.7416** |

Both: ReLU · Adam · early stopping. Tuned over `alpha` and `learning_rate_init`; best for both: $\alpha = 10^{-3}$, `lr` $= 0.005$.

### Key takeaways

- Depth doesn't help: 0.7405 vs 0.7416 (within noise).
- Tripling parameters bought nothing.
- Both NNs land below the classic learners.

# Combined Leaderboard




| Model | Val AUC |
|---|---|
| HistGradBoost | **0.7595** |
| Logistic Reg. | 0.7487 |
| Random Forest | 0.7470 |
| Deep MLP | 0.7416 |
| Shallow MLP | 0.7405 |

Tree boosting wins by $\sim$1 AUC point.

# Imbalance Handling on the Best MLP

| Method | Thresh. | AUC | PR-AUC | Prec. | Recall | F1 |
|---|---|---|---|---|---|---|
| Baseline | 0.500 | 0.742 | 0.217 | 0.00 | 0.00 | 0.00 |
| SMOTE | 0.500 | **0.648** | 0.134 | 0.16 | 0.21 | 0.18 |
| Threshold tuning $\star$ | 0.139 | 0.742 | 0.217 | 0.22 | **0.41** | **0.29** |

### Key takeaways

- Baseline at 0.5: AUC 0.742, recall **0%**.
- SMOTE hurts: AUC drops 0.09.
- Threshold tuning at 0.139 lifts recall to **41%**.

*Threshold > resampling.*

# Conclusions

### Best model overall

- **HGBC wins** (val AUC = **0.7595**) — nonlinear tree boosting captures feature interactions.
- **LR & RF** close behind — LR benefits from strong linear predictors (`EXT_SOURCE_*`).
- **Deep MLP** (0.7416) $\approx$ Shallow MLP (0.7405) — depth doesn't help.
- Both NNs lag classic shallow learners by $\sim$1–2 AUC pts.

### Imbalance handling on the MLP

- Threshold tuning lifts recall **0% $\rightarrow$ 41%**; SMOTE hurts AUC.
- Choosing the threshold matters more than rebalancing the data.

### Limitations

- Only main `application_*.csv` tables (auxiliary tables would add several AUC points).
- Single train/val split, not full $k$-fold CV.

# Recording

<<INSERT RECORDING LINK BEFORE SUBMISSION>>

*Minwoo Yoo · Nathaniel Badalov*